

Java Streams - Collectors

Numeric, Grouping, Partitioning & Joining — Detailed Notes

Prepared by Sushil Banka

1 Introduction

This document explains the purpose, functional interfaces, method signatures, examples, and use cases for important **terminal collectors** in Java Streams.

It covers:

- Numeric aggregation collectors:
 - `counting()`
 - `summarizingInt()`, `summarizingLong()`, `summarizingDouble()`
 - `summingInt()`, `summingLong()`, `summingDouble()`
 - `averagingInt()`, `averagingLong()`, `averagingDouble()`
 - `minBy()`, `maxBy()`
- Grouping and partitioning collectors:
 - `groupingBy()` (all overloads)
 - `groupingByConcurrent()` (all overloads)
 - `partitioningBy()` (all overloads)
- String concatenation collector:
 - `joining()` and its overloads

For each collector, the notes include:

- What the collector does
- Functional interface used
- Method signature of that interface
- Code example
- When to use it in real code

2 Numeric Collectors

2.1 counting()

What it does

Counts the number of elements in a stream.

Functional Interface Used

No functional interface is supplied by the user. Internally, the collector simply counts elements.

Example

```
long total = employees.stream()
    .collect(Collectors.counting());
```

When to Use

Use when you need:

- the total number of items in the stream, or
- the count of items inside each group when used as a downstream collector.

2.2 Summarizing Collectors

Collectors

`summarizingInt()`, `summarizingLong()`, `summarizingDouble()` provide **all basic numeric statistics** for a field:

- count
- sum
- min
- max
- average

Functional Interface Used

- `ToIntFunction<T>`
- `ToLongFunction<T>`
- `ToDoubleFunction<T>`

Method Signatures

```
int applyAsInt(T value);  
long applyAsLong(T value);  
double applyAsDouble(T value);
```

Example

```
IntSummaryStatistics stats = employees.stream()  
    .collect(Collectors.summarizingInt(Employee::getSalary));
```

Now you can access:

- stats.getCount()
- stats.getSum()
- stats.getMin()
- stats.getMax()
- stats.getAverage()

When to Use

Use when you want **all numeric statistics at once** for a field instead of calling separate collectors for sum, average, etc.

2.3 Summing Collectors

Collectors

`summingInt()`, `summingLong()`, `summingDouble()` return the **total sum** of a numeric field.

Functional Interface Used

Same functional interfaces as summarizing collectors:

- `ToIntFunction<T>`
- `ToLongFunction<T>`
- `ToDoubleFunction<T>`

Example

```
int totalSalary = employees.stream()  
    .collect(Collectors.summingInt(Employee::getSalary));
```

When to Use

Use when only the **sum** is required and you do not need min, max, or average.

2.4 Averaging Collectors

Collectors

`averagingInt()`, `averagingLong()`, `averagingDouble()` return the **average** as a double.

Functional Interface Used

Same as summing collectors:

- `ToIntFunction<T>`
- `ToLongFunction<T>`
- `ToDoubleFunction<T>`

Example

```
double avgSalary = employees.stream()
    .collect(Collectors.averagingInt(Employee::getSalary));
```

When to Use

Use when you only need the **average** of some numeric field, without full statistics.

2.5 minBy() and maxBy()

What They Do

- `minBy()` finds the element with the smallest value according to a comparator.
- `maxBy()` finds the element with the largest value according to a comparator.

Both return an `Optional<T>` because the stream might be empty.

Functional Interface Used

`Comparator<T>`

Comparator Method Signature

```
int compare(T a, T b);
```

Example – Minimum

```
Optional<Employee> minSalary =  
    employees.stream()  
        .collect(Collectors.minBy(  
            Comparator.comparing(Employee::getSalary)  
        ));
```

Example – Maximum

```
Optional<Employee> maxSalary =  
    employees.stream()  
        .collect(Collectors.maxBy(  
            Comparator.comparing(Employee::getSalary)  
        ));
```

When to Use

Use when you want the **entire object** (e.g., the full `Employee`) having:

- the smallest value of some field, or
- the largest value of some field.

Not just the min or max numeric value, but the whole element.

3 groupingBy — Multi-Group Classification

`groupingBy` divides elements into **multiple buckets** based on a **classifier function**. This behaves like SQL's `GROUP BY`, but performed inside Java code using Streams.

3.1 Why do we use it?

- To categorize elements into logical groups.
 - Example: group employees by department.
 - Example: group products by category.
- To run aggregations inside each group:
 - total salary per department,
 - number of employees per role,
 - max/min/average of some field per group.

- To convert a flat stream into a rich map:

`Map<Key, Value>`

where `Key` is the group key and `Value` can be a `List` or aggregated value.

3.2 Functional Interface Used

- **Classifier:** `Function<T, K>`
 - Input: element of type `T`.
 - Output: group key of type `K`.
 - This key decides in which bucket the element will go.
- **Downstream collectors (optional):** other `Collectors` like `counting()`, `summingInt()`, `mapping()`, etc.

3.3 `groupingBy(classifier)`

Syntax

`Collectors.groupingBy(classifier)`

Detailed Explanation

- Takes a **classifier function** (`Function<T, K>`).
- For each element:
 - It calls the classifier.
 - The returned key is used as the map key.
- If the key is seen for the first time:
 - A new bucket (entry in the `Map`) is created.
 - This bucket is a `List<T>`.
- If the key already exists:
 - The element is added to the existing list.
- No aggregation logic (sum, count, etc.) is applied. Only grouping.

Output Type

`Map<Key, List<T>>`

Example

```
Map<String, List<Employee>> byDept =  
    employees.stream()  
        .collect(Collectors.groupingBy(Employee::getDepartment));
```

This creates one list per department.

3.4 groupingBy(classifier, downstream)

Syntax

```
Collectors.groupingBy(classifier, downstreamCollector)
```

Detailed Explanation

- Still divides elements into groups using the **classifier**.
- But instead of collecting values as `List<T>`, it applies a **downstream collector** to each group.
- The downstream collector decides what the final value per group will be:
 - Could be a **Long** (e.g., `count`).
 - Could be a **Double** (e.g., `sum` or `average`).
 - Could be another collection (e.g., `list of names`).
- So, each bucket stores a **single aggregated value**, not a list of all elements.

Output Type

`Map<Key, AggregatedResult>`

Common Downstream Examples

- `counting()` — number of items per group.
- `summingInt()`, `summingDouble()` — sum of a numeric field.
- `averagingInt()`, `averagingDouble()` — average value per group.
- `maxBy()`, `minBy()` — highest or lowest element based on a comparator.
- `mapping(Function, toList())` — transform items first, then collect into a list.

Examples

```
// 1. Count employees per department
Map<String, Long> countByDept =
    employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
            Collectors.counting()));

// 2. Total salary per department
Map<String, Double> salaryByDept =
    employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
            Collectors.summingDouble(Employee::
                getSalary))));

// 3. List of names per department
Map<String, List<String>> namesByDept =
    employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
            Collectors.mapping(Employee::getName,
                Collectors.toList())));
```

3.5 groupingBy(classifier, mapFactory, downstream)

Syntax

```
Collectors.groupingBy(classifier, mapFactory, downstream)
```

Deep Explanation

- Same as the previous version, but with an extra parameter: `mapFactory`.
- `mapFactory` is a `Supplier<Map<K, D>>` that tells Java what type of map to create.
- Use this when:
 - you want sorted keys,
 - you want insertion order preserved,
 - you want a special map implementation.

Common Map Factories

- `TreeMap::new` — keys are sorted (natural ordering).
- `LinkedHashMap::new` — maintains insertion order.
- `HashMap::new` — default unordered map.

Output Type

CustomMap<Key, AggregatedResult>

Example

```
Map<String, Double> salaryByDeptSorted =
    employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
            java.util.TreeMap::new,
            Collectors.summingDouble(Employee::
                getSalary)));
```

Here, the resulting map is a `TreeMap`, so departments appear in sorted key order.

4 groupingByConcurrent — Multi-Group With Thread Safety

`groupingByConcurrent` is like `groupingBy`, but designed for **concurrent use**, especially with `parallelStream()`.

It internally uses a **ConcurrentMap** (usually `ConcurrentHashMap`).

4.1 When should we use it?

- When you are using `parallelStream()` on large datasets.
- When multiple threads will write to the grouping map at the same time.
- When you do not care about ordering of keys (`ConcurrentHashMap` is unordered).

4.2 Functional Interfaces Used

- `classifier` → `Function<T, K>`
- `mapFactory` → `Supplier<ConcurrentMap<K, D>>`
- `downstream` → `Collector<? super T, A, D>`

4.3 groupingByConcurrent(classifier)

Explanation

- Similar to `groupingBy(classifier)`.
- Uses a **ConcurrentMap** as the result instead of a normal `Map`.
- Good for parallel processing but:

- does not guarantee ordering,
- has more memory and concurrency overhead, but is safer for multi-thread writes.

Output Type

`ConcurrentMap<Key, List<T>>`

Example

```
ConcurrentMap<String, List<Employee>> byDeptConcurrent =  
    employees.parallelStream()  
        .collect(Collectors.groupingByConcurrent(Employee::getDepartment));
```

4.4 `groupingByConcurrent(classifier, downstream)`

Explanation

- Same idea as `groupingBy(classifier, downstream)`.
- Groups elements by classifier and applies a downstream aggregator.
- Safe and optimized for parallel streams.

Output Type

`ConcurrentMap<Key, AggregatedResult>`

Example

```
ConcurrentMap<String, Long> countByDeptConcurrent =  
    employees.parallelStream()  
        .collect(Collectors.groupingByConcurrent(Employee::getDepartment,  
                                                    Collectors.counting()));
```

4.5 `groupingByConcurrent(classifier, mapFactory, downstream)`

Explanation

- Allows choosing the concrete **concurrent map type**.
- Example map types:
 - `ConcurrentHashMap` — standard concurrent map.
 - `ConcurrentSkipListMap` — concurrent and sorted.

Output Type

`CustomConcurrentMap<Key, AggregatedResult>`

Example

```
ConcurrentMap<String, Double> salaryByDeptConcurrent =  
    employees.parallelStream()  
        .collect(Collectors.groupingByConcurrent(Employee::getDepartment,  
                                                    java.util.concurrent.  
                                                        ConcurrentHashMap::new,  
                                                    Collectors.summingDouble(Employee  
                                                        ::getSalary))));
```

5 partitioningBy — Boolean Split (True/False)

`partitioningBy` splits elements into exactly **two groups** based on a **predicate** (boolean condition).

5.1 How is it different from `groupingBy`?

- **`groupingBy`:**
 - Classifier returns any key type (String, Integer, Enum, etc.).
 - Number of groups = number of distinct keys (can be many).
- **`partitioningBy`:**
 - Predicate returns only **true** or **false**.
 - Number of groups is always exactly **2**: **true** group and **false** group.

5.2 Functional Interface Used

- **`predicate`** → `Predicate<T>`
 - Input: element T.
 - Output: boolean (true/false).

5.3 `partitioningBy(predicate)`

Explanation

- Evaluates the predicate for each element.
- If predicate is **true** → element goes to the **true** bucket.

- If predicate is **false** → element goes to the **false** bucket.
- Both keys **true** and **false** are present in the map (even if one list is empty).

Output Type

`Map<Boolean, List<T>>`

Example

```
Map<Boolean, List<Employee>> byHighSalary =
    employees.stream()
        .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000));
```

Here:

- **true** → employees with salary > 50000.
- **false** → employees with salary ≤ 50000.

5.4 partitioningBy(predicate, downstream)

Explanation

- Uses the same boolean-based split.
- Instead of storing `List<T>` per group, we apply a downstream aggregator.
- Each of the two partitions (true and false) gets its own aggregated value.

Output Type

`Map<Boolean, AggregatedResult>`

Examples

```
// 1. Count how many in each partition
Map<Boolean, Long> countByHighSalary =
    employees.stream()
        .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000,
                                           Collectors.counting()));

// 2. Sum of salary in each partition
Map<Boolean, Double> salaryByPartition =
    employees.stream()
        .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000,
                                           Collectors.summingDouble(Employee::
                                                                    getSalary))));
```

```
// 3. Names in each partition
Map<Boolean, List<String>> namesPartitioned =
    employees.stream()
        .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000,
            Collectors.mapping(Employee::getName,
                Collectors.toList())));
```

6 Collectors.joining() — Detailed Explanation

The `joining` collector is used to concatenate elements of a `Stream<String>` (or other `CharSequence`) into a single `String`.

6.1 Very Important: `joining()` is only for `CharSequence` types

The method signature of `joining` is:

```
public static Collector<CharSequence, ?, String> joining(...)
```

This means:

- The stream element type must implement `CharSequence`.
- So it supports only:
 - `String`
 - `StringBuilder`
 - `StringBuffer`
- It does **not** directly support:
 - `Integer`
 - `Double`
 - arbitrary objects (e.g., `Employee`, `Product`)

Conclusion: `Collectors.joining()` is strictly a string-based collector. To use it with non-string objects, we must convert them to strings first using `map()`.

6.2 Joining Non-String Streams

Even though `joining()` works only for strings, we can still join non-string objects by converting them to strings.

Example: Joining Integers

```
List<Integer> nums = List.of(1, 2, 3);

String result = nums.stream()
    .map(Object::toString) // convert Integer -> String
    .collect(Collectors.joining(", "));
```

Output:

```
1, 2, 3
```

Example: Joining Object Fields (e.g., Employee Names)

```
String names = employees.stream()
    .map(Employee::getName) // get String name
    .collect(Collectors.joining(" | "));
```

Output:

```
John | Emma | Robert
```

This works because we explicitly convert each stream element into a `String` before using `joining()`.

6.3 `joining()`

Meaning

Concatenates all stream elements without any separator.

Example

```
List<String> names = List.of("A", "B", "C");
String result = names.stream()
    .collect(Collectors.joining());
```

Output:

```
ABC
```

6.4 `joining(delimiter)`

Meaning

Inserts the given separator between elements.

Example

```
List<String> names = List.of("A", "B", "C");
String result = names.stream()
    .collect(Collectors.joining(", "));
```

Output:

```
A, B, C
```

6.5 joining(delimiter, prefix, suffix)

Meaning

Concatenates elements with a delimiter and wraps the entire result with prefix and suffix.

Example

```
List<String> names = List.of("A", "B", "C");
String result = names.stream()
    .collect(Collectors.joining(", ", "[", "]"));
```

Output:

```
[A, B, C]
```

6.6 Summary Table for joining()

Method	Description	Ex. Output
joining()	Concatenates directly without separator	"ABC"
joining(delimiter)	Adds a separator between elements	"A, B, C"
joining(delimiter, prefix, suffix)	Separator + wrap with prefix and suffix	"[A, B, C]"

6.7 When to Use joining()

- Creating CSV-like strings (e.g., "A,B,C").
- Creating bracketed lists like [A, B, C].
- Generating readable strings of names, IDs, etc.
- Building SQL IN clause strings:

```
('A','B','C')
```

after appropriate mapping and formatting.

7 Using Collectors Inside Grouping

Here is a real-world usage example combining grouping and numeric downstream collectors:

7.1 Example

```
Map<String, Double> result =
    transactions.stream()
        .collect(Collectors.groupingBy(
            Transaction::getUser,
            Collectors.summingDouble(Transaction::getAmount)
        ));
```

This:

- Groups by each user.
- Sums amounts inside each group.

This pattern is very common in real systems:

- total spend per user,
- total quantity per product,
- total marks per student, etc.

8 Summary Tables and Comparisons

8.1 Numeric Collectors Summary Table

Collector	Functional Interface	When to Use
counting()	None	Count items
summarizingInt()	ToIntFunction	Need full stats
summarizingLong()	ToLongFunction	Need full stats
summarizingDouble()	ToDoubleFunction	Need full stats
summingInt()	ToIntFunction	Only need sum
summingLong()	ToLongFunction	Only need sum
summingDouble()	ToDoubleFunction	Only need sum
averagingInt()	ToIntFunction	Only need average
averagingLong()	ToLongFunction	Only need average
averagingDouble()	ToDoubleFunction	Only need average
minBy()	Comparator	Need object with minimum value
maxBy()	Comparator	Need object with maximum value

8.2 Summary — Most Important Comparison

- **groupingBy**
 - Multiple groups (based on any key).
 - Uses normal `Map`.
 - By default stores `List<T>` per group.
 - Can apply downstream collectors to aggregate.
- **groupingByConcurrent**
 - Multiple groups stored in `ConcurrentMap`.
 - Designed for `parallelStream()`.
 - Thread-safe for concurrent updates.
- **partitioningBy**
 - Exactly two groups: `true` and `false`.
 - Uses `Predicate<T>` instead of `Function<T, K>`.
 - Very useful when the condition is naturally boolean (e.g., `isAdult`, `isActive`, `isHighSalary`).
- **joining**
 - Works only on `CharSequence` (Strings and similar).
 - Used to build a single combined String from a String stream.
 - For non-String objects, use `map()` to convert to String first.

9 Conclusion

These collectors provide powerful numerical aggregation and grouping features for Java Streams. Understanding the functional interfaces behind them helps in understanding how values are extracted and processed inside downstream collectors.

- Numeric collectors (`counting`, `summing`, `averaging`, `summarizing`, `minBy`, `maxBy`) help you derive metrics from your data.
- Grouping collectors (`groupingBy`, `groupingByConcurrent`) help you organize and aggregate data by keys, very similar to SQL `GROUP BY`.
- `partitioningBy` is perfect when your split is naturally boolean.
- `joining` is your go-to collector when you want to turn a stream of strings into a single formatted string.

With these notes, you can read any stream pipeline using collectors and understand exactly:

- what is being grouped,
- how it is aggregated,
- and what the final result structure will look like.